Projetando Estilos BIBTEX

Oren Patashnik

8 de fevereiro de 1988

5. Hacking no estilo bibliográfico

Este documento começa (e termina) com a Seção 5, porque, na realidade, é a seção final do "BibTEXing" [4], a documentação geral do BibTEX. Mas esse documento foi criado para todos os usuários do BibTEX, enquanto este é apenas para designers de estilo, portanto, os dois são fisicamente separados. Ainda assim, você deve estar completamente familiarizado com o "BibTEXing", e todas as referências neste documento a seções e números de seção pressupõem que os dois documentos sejam um só.

Esta seção, juntamente com o arquivo de documentação de estilo padrão btxbst.doc, deve explicar como modificar arquivos de estilo existentes e produzir novos. Se você é um hacker de estilo sério, deve estar familiarizado com van Leunen [7] para questões de estilo, com Lamport [3] e Knuth [2] para questões de formatação, e talvez com Scribe [6] para detalhes de compatibilidade. E, já que está aqui, se não leu o ótimo livrinho de Strunk e White [5], deveria pelo menos dar uma olhada em suas entradas no banco de dados e na lista de referências para ver como o BibTEX lida com múltiplos nomes.

Para criar um novo estilo, é melhor começar com um estilo existente próximo ao seu e, em seguida, modificá-lo. Isso vale mesmo se você estiver apenas atualizando um estilo antigo para a versão 0.99 do BibTEX (atualizei quatro estilos não padrão, então digo isso com alguma experiência). Se você quiser inserir em um novo estilo alguma função que você escreveu para um estilo antigo (versão 0.98i), lembre-se de que a ordem dos argumentos para a função de atribuição (`:=`) foi invertida. Quando terminar seu estilo, você pode tentar executá-lo em todo o banco de dados XAMPL.BIB para garantir que ele processe todos os tipos de entrada padrão.

Se você encontrar algum bug nos estilos padrão, ou se houver coisas que você gostaria de fazer com arquivos no estilo bibliográfico, mas não consegue, reclame com Oren Patashnik.

5.1. Descrição geral

Você escreve estilos bibliográficos em uma linguagem de pilha ***postfix***. Não é muito difícil descobrir como [se escreve] consultando a documentação do estilo padrão, mas esta descrição fornece alguns detalhes (e fornecerá mais detalhes se houver demanda).

Basicamente, o arquivo de estilo é um programa, escrito em uma linguagem sem nome, que informa ao BibTEX como formatar as entradas que irão para a lista de referências (daqui em diante, "as entradas" será "a lista de entradas" ou simplesmente "a lista", se o contexto permitir). Esta linguagem de programação possui dez comandos, descritos na próxima subseção. Esses comandos manipulam os objetos da linguagem: constantes, variáveis, funções, a pilha e a lista de entradas. (Aviso: A terminologia nesta documentação, escolhida para facilitar a explicação, é ligeiramente diferente da do BibTEX. Por exemplo, "variáveis" e "funções" desta documentação são ambas "funções" para o BibTEX. Tenha isso em mente ao interpretar as mensagens de erro do BibTEX.)

Existem dois tipos de funções: as **internas** fornecidas pelo BibTEX (descritas na Seção 5.3) e as **que você define** usando o comando MACRO ou FUNCTION.

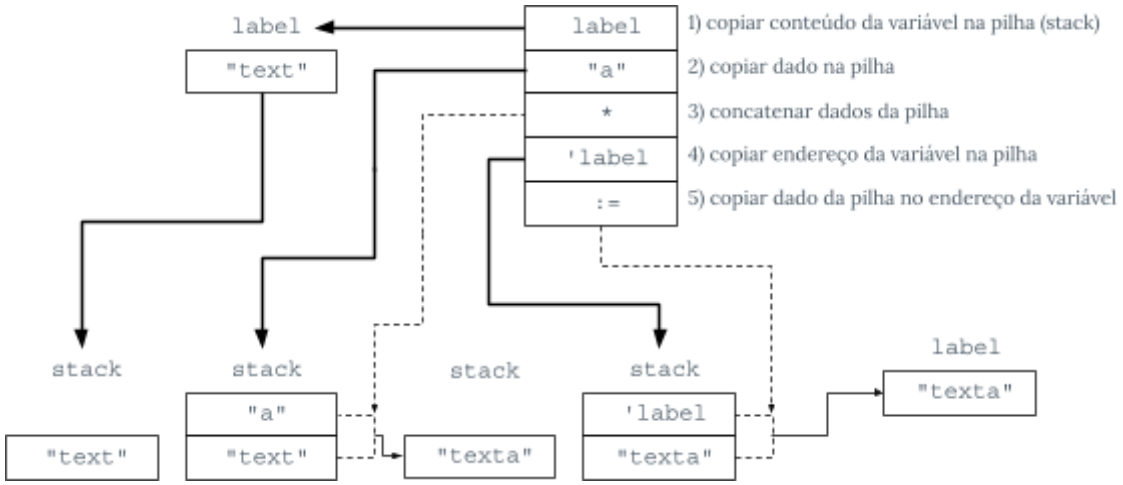
A tarefa que mais consumirá tempo, como designer de estilo, será criar ou modificar funções usando o comando FUNCTION (na verdade, familiarizar-se com as referências listadas acima consumirá mais tempo, mas suponha por enquanto que isso seja feito).

Vejamos um fragmento de função de exemplo. Suponha que você tenha uma variável de string chamada `label` e uma variável inteira chamada `lab.width`, e suponha que você queira acrescentar o caractere `"a"` a `label` e incrementar `lab.width`:

```
. . .
label "a" * 'label := % label := label * "a"
lab.width #1 + 'lab.width := % lab.width := lab.width + 1
. . .
```

Na primeira linha, o `label` empilha o **valor da variável**. Em seguida, o `"a"` empilha a **constante literal (string)** **`'a'`**. Em seguida, a função interna **`*`** desempilha as duas strings do topo e empilha sua **concatenação**. O `'label` empilha o **nome da variável**. E, por fim, a função interna **`:=`** empilha o **nome da variável** e a **concatenação**, realizando **a atribuição**. O BibTEX trata o conteúdo após o `%` como um comentário no arquivo de estilo. A segunda li-

nha é semelhante, exceto pelo fato de usar `#1`, sem espaços entre o '#' e o '1', para empilhar essa `constante inteira 1`.



O espaçamento não nulo aqui é arbitrário: vários espaços, tabulações ou novas linhas são equivalentes a um único (exceto que provavelmente é melhor não ter linhas em branco dentro dos comandos, como será explicado em breve).

Para **strings constantes**, absolutamente **qualquer caractere de impressão é válido** entre duas aspas duplas consecutivas, mas o **BibTEX** aqui (e somente aqui) **trata equivalentes em maiúsculas e minúsculas como diferentes**. Além disso, o espaçamento é relevante dentro de uma constante de string, e você não deve dividir uma constante de string entre linhas (ou seja, as aspas duplas iniciais e finais devem estar na mesma linha).

Os **nomes de variáveis** e **funções** não podem começar com um numeral e não podem conter nenhum dos dez caracteres restritos na página 143 do livro *LATEX*, mas pode conter quaisquer caracteres de impressão. Além disso, o **BibTEX considera equivalentes em maiúsculas e minúsculas como iguais**.

Inteiros e **strings** são os **únicos tipos de valor** para constantes e variáveis (booleanos são implementados simplesmente como inteiros 0 ou 1).

Existem três tipos de variáveis:

variáveis globais

Elas são de valor inteiro ou string, declaradas usando um comando `INTEGERS` ou `STRINGS`.

variáveis de entrada

São valores inteiros ou string, declarados usando o comando `ENTRY`.

Cada uma tem um valor para cada entrada na lista (exemplo: uma variável `label` pode armazenar a string de rótulo que você usará para a entrada).

campos

São variáveis somente leitura, com valor de string, que armazenam informações do arquivo de banco de dados; seus valores são definidos pelo comando **READ**. Assim como as variáveis de entrada, cada uma tem um valor para cada entrada.

5.2. Comandos

Existem dez comandos de arquivo de estilo: cinco (`ENTRY`, `FUNCTION`, `INTEGERS`, `MACRO` e `STRINGS`) declaram e definem variáveis e funções; um (`READ`) lê as informações do banco de dados; e quatro (`EXECUTE`, `ITERATE`, `REVERSE` e `SORT`) manipulam as entradas e produzem a saída. Embora os nomes dos comandos apareçam aqui em letras maiúsculas, o BibTEX ignora a diferença entre maiúsculas e minúsculas.

Algumas restrições: deve haver exatamente um comando `ENTRY` e um comando `READ`; o comando `ENTRY`, todos os comandos `MACRO` e certos comandos `FUNCTION` (veja a descrição de `call.type$` na próxima subseção) devem preceder o comando `READ`; e o comando `READ` deve preceder os quatro que manipulam as entradas e produzem a saída.

Também é melhor (mas não essencial) deixar pelo menos uma linha em branco entre os comandos e não deixar nenhuma linha em branco dentro de um comando; isso ajuda o BibTEX a se recuperar de quaisquer erros de sintaxe que você cometer.

Você deve colocar cada argumento de cada comando entre chaves. Consulte a documentação em estilo padrão para problemas sintáticos não descritos nesta seção.

Aqui estão os dez comandos:

ENTRY **Declara os campos e as variáveis de entrada**. Possui **três argumentos**, sendo cada um uma lista (possivelmente vazia) de nomes de variáveis. As três listas são: **campos**, **variáveis de entrada inteiras** e **variáveis de entrada literais** (string). Há um campo adicional que o BibTEX declara automaticamente, `crossref`, usado para referência cruzada. E há uma variável de entrada literal adicional declarada automaticamente, `sort.key$`, usada pelo comando `SORT`. Cada uma dessas

variáveis tem um valor para cada entrada na lista.

`INTEGERS` **Declara variáveis globais inteiras**. Possui **um argumento**: uma **lista de nomes de variáveis**. Existem duas dessas variáveis declaradas automaticamente, `entry.max$` e `global.max$`, usadas para limitar o comprimento de variáveis do tipo string. Você pode ter qualquer número desses comandos, mas a declaração de uma variável deve preceder seu uso.

`STRINGS` **Declara variáveis globais de string**. Possui **um argumento**: uma **lista de nomes de variáveis**. Você pode ter qualquer número desses comandos, mas a declaração de uma variável deve preceder seu uso.

`FUNCTION` Define uma nova função. Ela tem **dois argumentos**: o primeiro é **o nome da função** e o segundo é **sua definição**. Você deve definir uma função antes de usá-la; funções recursivas são, portanto, ilegais.

`MACRO` **Define uma macro de string**. Ela tem **dois argumentos**: o primeiro é **o nome da macro**, que é tratado como qualquer outro nome de variável ou função, e o segundo é **sua definição**, que **deve ser delimitada por aspas duplas**. Você deve ter uma para cada abreviação de três letras de mês; além disso, você deve ter uma para nomes de periódicos comuns. O banco de dados do usuário pode substituir qualquer definição que você definir usando este comando. Se você quiser definir uma string que o usuário não possa acessar, use o comando `FUNCTION`, que tem uma sintaxe compatível.

`READ` **Extrai** do arquivo de banco de dados **os valores de campo para cada entrada** na lista. Não possui argumentos. **Se uma entrada** de banco de dados **não tiver um valor para um campo** (e provavelmente nenhuma entrada de banco de dados terá um valor para cada campo), essa variável de campo **será marcada como ausente para a entrada**.

`EXECUTE` **Executa** uma única **função**. Possui **um argumento**: **o nome da função**.

`ITERATE` **Executa** uma única **função**, **uma vez para cada entrada na**

lista, na ordem atual da lista (inicialmente, a lista está em ordem de citação, mas o comando `SORT` pode alterar isso). Tem **um argumento**: **o nome da função**.

`REVERSE` Exatamente o mesmo que o comando `ITERATE`, exceto que **ele executa a função na lista de entradas na ordem inversa**.

`SORT` Classifica a lista de entradas usando os valores da variável de entrada literal `sort.key$`. Não tem argumentos.

5.3. As funções integradas

Antes de chegarmos às funções internas, algumas palavras sobre outros objetos internos. Há uma variável de entrada de string interna, `sort.key$`, que o programa de estilo deve definir se o estilo for fazer a classificação. Há um campo interno, `crossref`, usado para o recurso de referência cruzada descrito na Seção 4. E há duas variáveis globais inteiras internas, `entry.max$` e `global.max$`, que são definidas por padrão para algumas constantes internas do BibTEX; você deve truncar strings com esses comprimentos antes de atribuir às variáveis de string, para não gerar nenhuma mensagem de aviso do BibTEX.

Atualmente, existem **37 funções internas**. Todas as funções internas **terminam com uma letra `'$'`** no nome . A seguir, "primeiro", "segundo" e assim por diante referem-se à ordem de extração. Um "literal" é um elemento na pilha e pode ser um valor inteiro, uma string, um nome de variável ou função ou um valor especial que denota um campo ausente. Se algum literal extraído tiver um tipo incorreto, o BibTEX reclama e insere o inteiro 0 ou a string nula, dependendo se a função deveria inserir um inteiro ou uma string.

`>` Desempilha os dois literais superiores (inteiros), compara-os e insere o inteiro 1 se o segundo for maior que o primeiro, 0 caso contrário.

`<` Análogo.

`=` Desempilha os dois primeiros literais (ambos inteiros ou strings), compara-os e insere o inteiro 1 se forem iguais, 0 caso contrário.

`+` Desempilha os dois literais (inteiros) superiores e insere a soma deles.

`-` Desempilha os dois literais (inteiros) superiores e insere a diferença entre eles (o primeiro subtraído do segundo).

`*` Desempilha os dois literais (string) superiores, concatena-os (na ordem inversa, ou seja, a ordem em que foram enviados) e envia a string resultante.

`:=` Desempilha os dois literais superiores e atribui ao primeiro (que deve ser global ou variável de entrada) o valor do segundo.

`add.period$` Desempilha o literal (string) superior, adiciona um `'.'` (ponto) a ele se o último caractere diferente de `'}'` não é um `'.'`, `'?'` ou `'!'` e envia esta string resultante.

`call.type$` Executa a função cujo nome é o tipo de entrada de uma entrada. Por exemplo, se uma entrada for do tipo `book`, esta função executa a função `book`. Quando fornecida como argumento para o comando `ITERATE`, `call.type$` produz a saída para as entradas. Para uma entrada com tipo desconhecido, ela executa a função `default.type`. Portanto, você deve definir (antes do comando `READ`) uma função para cada tipo de entrada padrão, bem como uma função `default.type`.

`change.case$` Desempilha os dois literais (string) superiores; altera a caixa do segundo (maiúsculo ou minúsculo, caixa alta ou baixa) de acordo com as especificações do primeiro, como segue. (Observação: a palavra "letras" na próxima sentença refere-se apenas àquelas no nível de chave 0, o nível de chave mais alto; nenhum outro caractere é alterado, exceto talvez os "caracteres especiais", descritos na Seção 4.) Se o primeiro literal for a string 't', ele converte todas as letras para minúsculas, exceto o primeiro caractere da string, que ele deixa sozinho, e exceto o primeiro caractere após qualquer dois pontos e, em seguida, o espaço em branco não nulo, que ele também deixa sozinho; se for a string 'l' (de lower), ele converte todas as letras para minúsculas; e se for a string 'u' (de upper), ele converte todas as letras para maiúsculas. Em seguida, insere a string resultante [na pilha]. Se qualquer um dos tipos estiver incorreto, ele reclama e insere a string nula; no entanto, se ambos os tipos estiverem corretos, mas a string de especificação (ou seja, a primeira string) não for uma das válidas, ele simplesmente insere a segunda de volta na pilha, após reclamar. (Outra observação: a fun-

ção ignora as diferenças entre maiúsculas e minúsculas na string de especificação; por exemplo, as strings 't' e 'T' são equivalentes para os propósitos desta função interna.)

chr.to.int$ Desempilha o literal (string) superior, garante que seja um único caractere, converte-o no inteiro ASCII correspondente e insere esse inteiro.

cite$ Empilha a string que foi o argumento do comando `\cite-` para esta entrada.

duplicate$ Desempilha o literal do topo da pilha e envia duas cópias dele.

empty$ Desempilha o literal superior e insere o inteiro 1 se for um campo ausente ou uma string sem caracteres que não sejam espaços em branco, 0 caso contrário.

format.name$ Desempilha os três literais superiores (uma string, um inteiro e um literal de string). O último literal de string representa uma lista de nomes (cada nome correspondendo a uma pessoa), o literal inteiro especifica qual nome escolher dessa lista e o primeiro literal de string especifica como formatar esse nome, conforme explicado na próxima subseção. Por fim, esta função insere o nome formatado.

if$ Desempilha os três primeiros literais (são dois literais de função e um literal inteiro, nessa ordem); se o inteiro for maior que 0, ele executa o segundo literal, caso contrario, ele executa o primeiro.

int.to.chr$ Desempilha o literal superior (inteiro), interpretado como o valor inteiro ASCII de um único caractere, converte-o na string de caractere único correspondente e insere essa string.

int.to.str$ Desempilha o literal superior (inteiro), converte-o em seu equivalente de string (único) e insere essa string.

missing$ Desempilha o literal superior e insere o inteiro 1 se for um campo ausente, 0 caso contrário.

newline$ Grava no arquivo bbl o que está acumulado no buffer de saída. Ele grava uma linha em branco se, e somente se, o buffer de saída estiver vazio. Como `write$` realiza que-

bras de linha razoáveis, você deve usar esta função somente quando quiser uma linha em branco ou uma quebra de linha explícita.

num.names$ Desempilha o literal (string) superior e insere o número de nomes que a string representa — um (1) mais o número de ocorrências da substring `"and"` (ignorando diferenças entre maiúsculas e minúsculas) cercada por espaço em branco não nulo no nível da chave superior.

pop$ Desempilha o topo da pilha, mas não a imprime; isso elimina um literal de pilha indesejado.

preamble$ Empilha a concatenação de todas as strings `@PREAMBLE` lidas dos arquivos de banco de dados.

purify$ Desempilha o literal superior (string), remove caracteres não alfanuméricos, exceto caracteres de espaço em branco, hifens e laços (todos eles são convertidos em um espaço), remove certos caracteres alfabéticos contidos nas sequências de controle associadas a um "caractere especial" e insere a string resultante.

quote$ Empilha a string que consiste no caractere de aspas duplas.

skip$ Não é uma operação.

stack$ Desempilha e imprime a pilha inteira; é destinado a ser usado por designers de estilo durante a depuração.

substring$ Desempilha os três literais superiores (são os dois literais inteiros `len` e `start`, e um literal de string, nessa ordem). Ele insere a substring de (no máximo) `len` caracteres consecutivos, começando no caractere de posição `start` (assumindo indexação de base 1) se `start` for positivo, e terminando no caractere de posição `start` a partir do final se `start` for negativo (onde o primeiro caractere a partir do final é o último caractere).

swap$ Troca os dois literais superiores na pilha.

text.length$ Desempilha o literal superior (string) e insere o número de caracteres de texto que ele contém, onde um caractere acentuado (mais precisamente, um "caractere especial", definido na Seção 4) conta como um único caractere de texto, mesmo que esteja faltando a chave correta

correspondente, e onde chaves não contam como caracteres de texto.

text.prefix$ Desempilha os dois literais superiores (o literal inteiro `len` e um literal string, nessa ordem). Ele remove a substring de (no máximo) `len` caracteres de texto consecutivos, começando do início da string. Esta função é semelhante a `substring$`, mas considera um "caractere especial", mesmo que não tenha a chave correta correspondente, como um único caractere de texto (em vez de quantos caracteres ASCII ele realmente contém), e esta função não considera chaves como caracteres de texto; além disso, esta função adiciona quaisquer chaves corretas correspondentes necessárias.

top$ Desempilha e imprime o topo da pilha no terminal e no arquivo de log. Útil para depuração.

type$ Empilha o tipo da entrada atual (`book`, `article`, etc.); se o tipo for desconhecido ou indefinido, empilha uma string nula.

warning$ Desempilha o literal (string) superior e o imprime após uma mensagem de aviso. Isso também incrementa a contagem do número de mensagens de aviso emitidas.

while$ Desempilha os dois literais (de função) do topo e continua executando o segundo enquanto o literal (inteiro) deixado na pilha pela execução do primeiro for maior que 0.

width$ Desempilha o literal superior (string) e insere o inteiro que representa sua largura em algumas unidades relativas (atualmente, centésimos de ponto, conforme especificado pela versão de junho de 1987 da fonte cmr10; o único caractere de espaço em branco com largura diferente de zero é o espaço). Esta função assume o literal literalmente; ou seja, assume que cada caractere na string deve ser impresso como está, independentemente de ter ou não um significado especial para TEX, exceto que "caracteres especiais" (mesmo sem as chaves corretas) são tratados de forma especial. Isso deve ser usado para comparar larguras de strings de rótulo.

write$ Desempilha o literal (string) superior e o grava no buffer

de saída (o que resultará em coisas sendo gravadas no arquivo bbl quando o buffer ficar cheio).

Observe que as funções internas `while$` e `if$` requerem dois literais de função na pilha. Você os obtém colocando imediatamente antes do nome de uma função uma aspa simples; caso não queira definir uma nova função com o comando `FUNCTION`, pode-se colocar simplesmente sua definição (ou seja, fornecendo o que seria o segundo argumento para o comando `FUNCTION`, incluindo as chaves circundantes). Por exemplo, o fragmento seguinte de função anexa o caractere `'a'` se a variável de string chamada `label` for não nula:

```
. . .
label "" =                    % the condition
   'skip$                     % built-in function
   { label "a" * 'label := } % function definition as is
if$
. . .
```

Uma função cujo nome você cita não precisa ser incorporada como `skip$` acima — pode, por exemplo, ser um nome de campo ou uma função que você definiu anteriormente.

5.4. Formatação de nomes

O que há em um nome? A Seção 4 descreve isso de forma bem resumida. Cada nome consiste em quatro partes: nome, preposição (*von*, de, etc.), sobrenome e sufixo (Jr, Filho, Neto, etc.); cada uma consiste em uma lista de tokens de nome, e qualquer lista, exceto a do sobrenome, pode estar vazia para um nome não nulo. Esta subseção descreve a string de formato que você deve fornecer à função interna `format.name$`.

Vejamos um exemplo de um nome muito longo. Suponha que uma entrada de banco de dados [1] tenha o campo

```
autor = "Charles Louis Xavier Joseph de la Vall{\'e}e Poussin"
```

e suponha que você queira este formato "sobrenome vírgula iniciais". Se você usar a string de formato

```
"{vv~}{ll}{, jj}{, f}?" % "{von}{last}{, Jr}{, first}"
```

O BibTEX produzirá

```
de~la Vall{\'e}e~Poussin, C.~L. X.~J?
```

como a string formatada.

Vamos analisar este exemplo em detalhes. Há quatro partes de nível 1 de chaves nesta string de formatação, uma para cada parte de um nome. Se a parte correspondente de um nome não estiver presente (a parte Jr para este nome), todo o conteúdo dessa parte será ignorado. Qualquer coisa no nível 0 de chaves é impressa literalmente (o erro de digitação presumido "?" para este nome está no nível 0 de chaves), mas você provavelmente não usará muito esse recurso.

Dentro de cada parte, uma letra dupla diz ao BibTEX para usar tokens inteiros e uma única letra para abreviá-los (essas letras devem estar no nível de chaves 1); todo o resto dentro da parte é usado literalmente (bem, quase tudo — continue lendo).

A ligação no final da parte "de" (em `{vv~}`) é uma ligação discricionária — o BibTEX emitirá uma ligação nesse ponto se achar necessário; caso contrário, emitirá um espaço. Se você realmente quiser uma ligação nesse ponto, independentemente do que o BibTEX pense, use duas delas (apenas uma será emitida); ou seja, use `{vv~~}`.

Um empate é discricionário somente se for o último caractere do pedaço; em qualquer outro lugar, ele é tratado como um caractere comum.

O BibTEX coloca strings padrão entre os tokens de uma parte do nome: para tokens inteiros, ele usa um espaço ou uma gravata, dependendo do que achar melhor, e para tokens abreviados, ele usa um ponto final seguido por um espaço ou uma gravata. No entanto, ele não usa essa string padrão após o último token em uma lista; portanto, não há ponto final após o "J" em nosso exemplo. Você deverá usar

```
"{vv~}{ll}{, jj}{, f.}"
```

para que o BibTEX produza a mesma string formatada, mas com o ponto de interrogação substituído por um ponto final. Observe que o ponto final deve ficar dentro da parte do primeiro nome (em `{, f}`), e não onde estava o ponto de interrogação, caso o nome não tenha a parte do primeiro nome.

Se você quiser substituir as strings padrão entre tokens do BibTEX, deverá especificar explicitamente uma string. Por exemplo, suponha que você queira que um rótulo contenha a primeira letra de cada token nas partes "de" e "sobrenome", sem espaços; você deve usar a string de formato

```
"{v{}}{l{}}"
```

para que o BibTEX produza `'dlVP'` como string formatada. Você deve fornecer uma string para cada trecho cujo padrão deseja substituir (o exemplo aqui usa a string nula para ambos os trechos), e essa string deve seguir ime-

diatamente a letra simples ou dupla do trecho. Você não pode ter nenhuma outra letra no nível 1 da chave na string de formato.

Referências

[1] Charles Louis Xavier Joseph de la Vall´ee Poussin. A *strong form of the prime number theorem*, século XIX.

[2] Donald E. Knuth. *The TEXbook*. Addison-Wesley, 1984.

[3] Leslie Lamport. *LATEX: A Document Preparation System*. Addison-Wesley, 1986.

[4] Oren Patashnik. *BibTEXing*. Documentation for general BibTEX users, 8 de fevereiro de 1988.

[5] William Strunk, Jr. e EB White. *The Elements of Style*. Macmillan, terceira edição, 1979.

[6] Unilogic, Ltd., Pittsburgh. *Scribe Document Production System User Manual*, abril de 1984. O capítulo doze e os apêndices E8 a E10 tratam de bibliografias.

[7] Mary-Claire van Leunen. A *Handbook for Scholars*. KNOPF, 1979.